

Aula 07 — Expressões, Atribuição e Controle

Capa

Abertura da Aula 07 de Paradigmas, sobre como as linguagens avaliam expressões, tratam atribuições e controlam o fluxo do programa. A base é o Sebesta (11ª ed., capítulos 7 e 8).

Objetivo e ligação com o mundo real

A ideia é entender como a linguagem avalia o que a gente escreve, para o código fazer exatamente o que parece fazer. Na prática: ler código dos outros sem cair em pegadinha de precedência ou endentação, evitar conversões implícitas perigosas (como o `==` do JS e do PHP) e escolher a estrutura de controle mais legível. Três casos reais mostram o custo disso: o `goto fail` da Apple (2014), o Zune travando em ano bissexto (2008) e o `==` do PHP aceitando hashes diferentes como iguais.

Roteiro da aula (80 min)

A aula é dividida em seis blocos: quiz de abertura (10 min), expressões (20), conversões e curto-circuito (15), atribuição (10), seleção, iteração, `goto` e comandos protegidos (15) e, por fim, o exercício de achar o bug (10). É uma aula exploratória: a turma prevê a saída antes de rodar, tudo no `onecompiler.com`.

Quiz de abertura: o que cada linha imprime?

Seis trechos para prever a saída. Resolvendo um por um:

- 1 • **Python** — `-4 512`. O `**` tem precedência sobre o menos unário, então é `-(2**2)`. E ele associa pela direita: `2 ** (3 ** 2) = 2 ** 9 = 512`.
- 2 • **Java** — `3345`. Primeiro `1 + 2 = 3` (inteiros). Ao encontrar a string `"3"`, tudo vira concatenação: `"33"`, `"334"`, `"3345"`.
- 3 • **JavaScript** — `3 52`. O `-` só existe para números, então `"5"` vira 5. Já o `+` com string concatena.
- 4 • **C** — `0. 3 > 2` vale 1 e depois `1 > 1` é falso.
- 5 • **Python** — padrão `True`. Como `0` é falso, o `or` devolve o segundo operando; e `3 > 2 > 1` é encadeado: `(3 > 2) and (2 > 1)`.
- 6 • **JavaScript** — `true false true`. A igualdade com coerção não é transitiva: `0 == ""` e `0 == "0"`, mas `"" != "0"`.

Expressões aritméticas: questões de projeto

Lista as perguntas que quem projeta uma linguagem precisa responder: precedência, associatividade, ordem de avaliação dos operandos, efeitos colaterais, sobrecarga e mistura de tipos. O exemplo `a + b * c` com `a=3`, `b=4`, `c=5` mostra o impacto: da esquerda para a direita dá `(3+4)*5 = 35`; com precedência (ou da direita para a esquerda) dá `3+20 = 23`. O APL avalia sempre da direita para a esquerda, então `3 × 4 + 5` vira `3 × 9 = 27`. Já o Lisp dispensa precedência, pois usa notação prefixa e os parênteses mandam.

Precedência e associatividade na prática

Tabela comparando Python, JavaScript e Java. `-2 ** 2` dá `-4` no Python e é erro de sintaxe no JS (precisa escrever `(-2) ** 2`); no Java não existe `**`. `2 ** 3 ** 2` dá `512` porque `**` associa pela direita. `1 + 2 + "3"` dá `TypeError` no Python e `"33"` em JS e Java. A divisão `-7 / 2` dá `-4` (com `//`, arredonda para baixo) no Python, `-3.5` no JS e `-3` no Java (trunca). E `-7 % 2` dá `1` no Python e `-1` nos outros dois. No Excel, `=-2^2` dá `4`. A moral do slide: na dúvida, use parênteses.

Ordem dos operandos e efeitos colaterais

O exemplo do livro: `a = a + fun1()`, onde `fun1` muda `a` para 17 e retorna 3. Se `a` for lido antes da chamada, dá `5 + 3 = 8`; se a chamada vier antes, dá `17 + 3 = 20`. O Java garante a ordem da esquerda para a direita (8). Em C o padrão não define, e o gcc 14 dá 20; o Go também não define e deu 20. O slide termina com a transparência referencial: sem efeitos colaterais, a ordem não importa.

Operadores sobrecarregados

Um mesmo símbolo com vários significados: o `+` soma números e concatena strings no Java; o `&` em C++ é E bit a bit e também "endereço de". No Java, `'a' + 1` dá 98 e `"" + 'a' + 1` dá `a1`. O Java não deixa o usuário sobrecarregar operadores (por isso `BigDecimal` usa `add` e `multiply`), mas o Python deixa, com métodos especiais como `__add__` e `__mul__`: `Vetor(1,2) + Vetor(3,4)` dá `Vetor(4, 6)` e multiplicar por 3 dá `Vetor(12, 18)`.

Conversões: alargamento, estreitamento e coerção

Alargamento (int para float) pode perder precisão: 123456789 vira 123456792 ao passar por float. O estreitamento exige cast: `(int) 3.99` dá 3 e `(byte) 200` dá -56. Em Java, `byte + byte` vira int, então `byte s = b + c` não compila. Também `1/2` é 0 e `1/2.0` é 0.5. A tabela mostra que C, Java e Python fazem coerção, enquanto Go, Rust, ML e F# exigem conversão explícita. O risco da coerção é esconder erros, como digitar a variável errada e o compilador aceitar mesmo assim.

Coerção implícita: JavaScript e PHP

A galeria de estranhezas do JS: `"5" - 2` dá 3, `"5" + 2` dá "52", `true + 1` dá 2, `[] + {}` dá "[object Object]" e `"b" + "a" + +"a" + "a"` dá "baNaNaN". No PHP ("type juggling"), `"0e123" == "0e456"` é verdadeiro porque as duas strings são lidas como número 0 em notação científica; é por isso que dois hashes md5 que começam com 0e "batem". Regra prática: em JS e PHP, compare com `===` e `!==`.

Expressões relacionais e booleanas

Em C o booleano é um inteiro: `a > b > c` com 3, 2, 1 dá 0, pois `(3 > 2)` vale 1 e `1 > 1` é falso; o correto é `(a > b) && (b > c)`, que dá 1. O Python encadeia comparações, então `a > b > c` é `True`. Em Python, JS e C, vários valores contam como falso (0, vazio, nulo), enquanto Java, C#, Go e Rust só aceitam booleano de verdade no `if`.

Avaliação em curto-circuito

O `&&` para assim que o resultado já está decidido, o que protege a busca do livro de ler `lista[lista.length]`. No Java, `s != null && s.length() > 0` dá `false`, mas com `&` simples estoura `NullPointerException`. No JS, `?.` evita erro em valor nulo, e `??` só troca `null` e `undefined`: com `qtd = 0`, `qtd || 10` dá 10 (o zero se perde) e `qtd ?? 10` dá 0. Cuidado também com efeitos colaterais: em `(a > b) || (b++ / 3)`, o `b++` só roda se `a <= b`.

Sentenças de atribuição

Em C, atribuição é uma expressão: dá para fazer `sum = count = 0` e `while ((ch = getchar()) != EOF)`. O lado ruim é que `if (x = y)` compila, atribuindo e testando `y`. A tabela mostra quem permite isso (C, C++, JS), quem exige booleano no `if` (Java, C#) e quem simplesmente proíbe (Python, Go, Rust). O Python tem atribuição múltipla (`a, b = b, a` troca sem variável temporária) e o operador morsa `:=`. Nas linguagens funcionais, `val` e `let` apenas dão nome a um valor que não muda.

Atribuição de modo misto

O Java só aceita alargamento: `double d = 3` pode, `int i = 3.7` não, e `(int) 3.7` dá 3. A pegadinha é `i += 3.7`, que compila por causa de um cast escondido: com `i = 3`, dá 6 ($3 + 3.7 = 6.7$, truncado). Em C a coerção vai para qualquer lado: `int i = 3.7` vira 3, `char c = 300` vira 44 e `unsigned u = -1` vira 4294967295. Armadilha clássica: `acertos / total * 100` com inteiros dá 0.0, porque a divisão inteira acontece antes do alargamento (cai de novo no exercício 6).

Estruturas de controle: quantas são necessárias?

Sequência, seleção e iteração. Böhm e Jacopini (1966) provaram que essas três bastam para qualquer algoritmo, então o `goto` não é necessário. As outras (`for`, `switch`, `do`, `foreach`) existem por facilidade de escrita, mas quanto maior a linguagem, menos do código alheio você reconhece.

Seleção de dois caminhos: o "else pendente"

No exemplo em C, a indentação sugere que o `else` pertence ao primeiro `if`, mas ele pertence ao `if` mais próximo que ainda não tem `else` (o de `count == 0`). Como `sum == 0` é verdadeiro e `count == 0` é falso, o `else` roda e `result` vira 1. Perl, Go e Rust evitam o problema exigindo chaves. O slide ainda mostra o `if` como expressão que devolve valor: no Rust (`let y = if ... else ...`) e no Python (`x if x > 0 else 2 * x`).

Seleção múltipla: do switch de C ao match

No `switch` de C, sem `break` o controle "cai" para o próximo caso: com `index = 1`, ele soma em odd, em even e ainda imprime `Error`. Linguagens modernas corrigem isso: o Java 14+ tem `switch` com `->` (sem queda e usável como expressão), o Rust tem `match` exaustivo (sem o `_` não compila), o Go termina cada caso sozinho (cair exige `fallthrough`), o C# proíbe queda implícita e o Python 3.10 ganhou `match`.

Laços controlados por contador

Em C, o `for` é na verdade um laço lógico disfarçado: mexer em `i` dentro do corpo altera a contagem, e o exemplo imprime 0 3 6 9 (cada volta soma 2 no corpo e 1 no incremento). No Python, o `for` percorre um iterável, então `i += 2` não muda nada e sai 0 1 2 3 4 5 6 7 8 9; já `range(10, 0, -3)` gera 10 7 4 1. As questões de projeto são tipo e escopo da variável, se pode alterá-la no corpo e se os parâmetros são avaliados uma vez ou a cada volta.

Laços lógicos, saídas e iteradores

O `break` rotulado do Java sai dos dois laços de uma vez. No JS, `for...in` percorre as **chaves** ("0", "1", "2"), por isso o total dá "0012", enquanto `for...of` percorre os **valores** e dá 60. O bug do Zune: em ano bissexto com `dias == 366`, nenhuma condição muda `dias` nem `ano`, e o laço nunca termina. E lembrete: o `do-while` roda ao menos uma vez.

Desvio incondicional: o goto

No `goto fail` da Apple, o segundo `goto fail`; está duplicado e sem condição: ele sempre desvia, a verificação da assinatura nunca roda e `err` continua 0, ou seja, "certificado válido". Dijkstra já criticava o `goto` em 1968. Ele existe em C, C++, C# e Go, mas não no Java. Os gotos restritos (`break`, `continue`, `return`) melhoram a legibilidade, e chaves obrigatórias ou `gcc -Wall` teriam pego o bug.

Comandos protegidos de Dijkstra

Cada ramo tem uma guarda e todas são avaliadas. Se várias forem verdadeiras, a escolha é não determinística (no exemplo do máximo, se $x == y$ qualquer ramo serve); se nenhuma for, é erro no `if` ou fim do laço no `do ... od`. A ideia inspirou o CSP de Hoare e o `select` do Go: rodando 1000 vezes com os dois canais prontos, ele sorteia algo como 505 a 495.

Exercício: encontre o bug (1/3)

1 · JavaScript

Saída: Total: 0012.

Regra: `for...in` percorre as chaves ("0", "1", "2") e não os valores, e `0 + "0"` vira concatenação de string.

Correção: usar `for...of`, que dá Total: 60.

```
for (const p of precos)
    total += p;
```

2 · Python

Saída: 180.0, e o esperado era 200.0.

Regra: `d or 10` trata 0 como falso (valor falsy), então o desconto 0 vira 10.

Correção: testar explicitamente se é None.

```
if d is None:
    d = 10
```

Exercício: encontre o bug (2/3)

3 · Java

Saída: inválido.

Regra: sem `break`, o `switch` cai de caso em caso: com `dia = 2`, passa por "segunda", "terça" e termina no `default`.

Correção: colocar `break` em cada caso, ou usar o `switch` com `->`, que dá segunda.

```
String nome = switch (dia) {
    case 1 -> "domingo";
    case 2 -> "segunda";
    case 3 -> "terça";
    default -> "inválido";
};
```

4 · C

Saída: saldo insuficiente e depois fim (as duas linhas, mesmo com saldo suficiente).

Regra: o `;` logo depois do `if` é um corpo vazio, então o `printf` seguinte roda sempre.

Correção: tirar o `;` e usar chaves.

```
if (saque > saldo) {
    printf("saldo insuficiente\n");
}
```

Exercício: encontre o bug (3/3)

5 · Go

Saída: dentro: 20 e depois fora: 10.

Regra: o `:=` declara uma **nova** variável `x` dentro do `if` (sombreamento), e o `x` de fora nunca muda.

Correção: usar `=` em vez de `:=`, para atribuir ao `x` existente; a saída vira dentro: 20 e fora: 20.

```
x = x * 2
```

6 · Java

Saída: 0.0%.

Regra: `acertos / total` é divisão inteira (`7/10 = 0`), e só depois o resultado vira `double`.

Correção: forçar a divisão real, por exemplo multiplicando por `100.0`; a saída vira `70.0%`.

```
double taxa = acertos * 100.0 / total;
```

Síntese: armadilhas e como as linguagens ajudam

O quadro-resumo junta tudo: para cada armadilha há uma proteção em alguma linguagem.

Precedência pede parênteses; efeitos colaterais na ordem dos operandos pedem ordem fixa (Java) e funções puras; a coerção silenciosa é barrada por conversão explícita (Go, Rust) e por `===`; booleanos confusos pedem `if` só com booleano; `if (x = y)` some quando atribuição não é expressão; `switch` sem `break` e `else` pendente são resolvidos por `->`, `match` exaustivo e chaves obrigatórias; `for ... in` por `for ... of`, iteradores e `break` rotulado; e o `goto` solto por `break`, `continue` e `return`.

Atividades complementares (autopesquisa)

Pesquisas sugeridas: expressões em Ruby e Lisp, expressões condicionais, tabelas de desvio na implementação do `switch`, laços por recursão em linguagens funcionais, iteradores e blocos do Ruby, e a leitura do artigo de Dijkstra (1968). A próxima aula é Subprogramas (capítulos 9 e 10).

A entrega sugerida é o mesmo exemplo de seleção em C (`switch`), Java (`switch` com `->`), Go (`switch`) e Rust (`match`), comparando queda entre casos, exaustividade e legibilidade.